

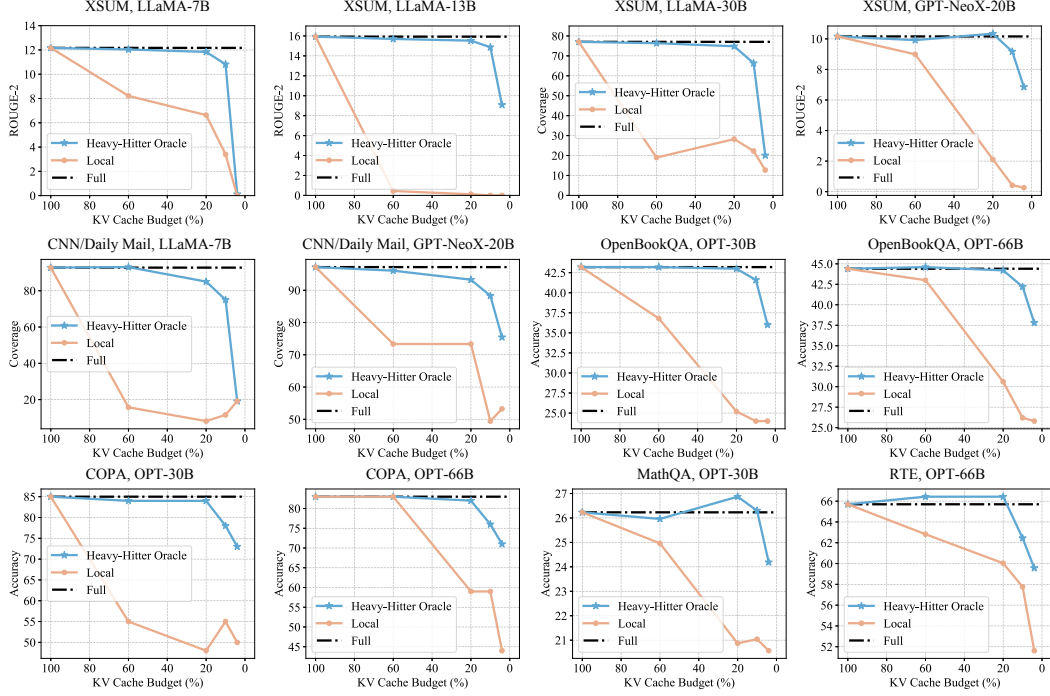


Figure 4: Comparison results between the baseline model with full cache, our H₂O, and the "Local" strategy that utilizes the most recent KV embeddings.

5 Empirical Evaluation

In this section, our goal is to demonstrate that H₂O, a remarkably simple KV cache eviction policy is capable of enhancing end-to-end throughput and reducing latency in wall-clock while maintaining generation quality across a broad spectrum of domains and tasks.

- In Section 5.1, we show that H₂O can reduce the memory footprint of KV cache by up to $5\times$ without accuracy degradation on a wide range of model architectures (OPT, LLaMA, GPT-NeoX), sizes (from 6.7B to 175B) and evaluation benchmarks (HELM and lm-eval-harness). More importantly, can enhance the performance of existing KV cache sparsification techniques.
- In Section 5.2, we demonstrate that H₂O can increase the inference throughput by up to $3\times$, $29\times$, $29\times$ compared to the state-of-the-art inference engine FlexGen, DeepSpeed and the widely used Hugging Face Accelerate without compromising model quality.
- In Section 5.3, we present extensive ablation studies to show the effectiveness of H₂O under different sequence lengths, especially the input with infinite sequence length and its compatibility with quantization.

All details (hyperparameters, data splits, etc.), along with additional experiments, are in Appendix A.

5.1 End-to-End Results

We demonstrate that H₂O can reduce KV cache memory footprint by $5\text{-}10\times$ while achieving comparable accuracy on a majority of tasks.

Setup. Our experiments are based on three representative model families of LLMs, including the OPT [39] with model sizes, LLaMA [40], and GPT-NeoX-20B [41]. We sample eight tasks from two popular evaluation frameworks (HELM [16] and lm-eval-harness [15]): COPA [42], MathQA [43], OpenBookQA [44], PiQA [45], RTE [46], Winogrande [47], XSUM [48], CNN/Daily Mail [49]. Also, we evaluate our approach on recent generation benchmarks, AlpacaEval [50] and MT-bench [51], and the details are included in Appendix. We use NVIDIA A100 80GB GPU.

Baselines. Since H₂O evenly assigns the caching budget to H₂ and the most recent KV, except for full KV cache, we consider the "Local" strategy as a baseline method. In addition, we also provide two different variants of Sparse Transformers (strided and fixed) as strong baselines. Also, the full

KV cache with fewer shots (0/1-shot) prompts are considered as the baseline, which has a similar sequence length of the 5-shot tasks with 20% KV cache budget.

Main Results. We evaluate LLMs with KV cache budget ranging from 4% to 100% on 5-shot downstream tasks. Results are summarized in Figure 4 and Table 1& 2. The following observations can be drawn: (1) With different KV cache budgets, our H₂O demonstrates consistent and significant improvements against the "Local" strategy across various model sizes, model types, and downstream tasks. We can draw similar conclusions comparing H₂O with other baselines like Sparse Transformer; (2) Meanwhile, with less than 20% KV cache budget (*i.e.*, more than 5× memory reduction), H₂O achieves comparable performance as the model with full KV embeddings; (3) H₂O with 20% KV cache budget approximately uses 1.2 samples per input and show consistent improvement over zero-shot and one-shot full model that use 1 and 2 samples, respectively. (4) Our H₂O shows consistent effectiveness in the more challenging long sequence generation tasks, XSUM, and CNN/Daily Mail.

Table 1: Quantatively comparison between H₂O with Full methods of different number of shots.

Methods	PiQA	COPA	OpenbookQA	Winogrande
Full	80.09	81.00	44.80	71.51
0-shot Full	78.89	76.00	41.40	70.00
1-shot Full	79.11	76.00	43.60	70.24
Local	57.94	56.00	28.40	51.30
H ₂ O	79.22	85.00	43.80	71.67

Analysis. Since the evicted KV will not be seen in the future steps, dropping certain critical KV embeddings can cause a severe functional collapse, resulting in significant performance degradation, *e.g.*, in {LLaMA-13B, XSUM} {LLaMA-7B, CN-N/Daily Mail}, the "Local" strategy collapses at 60% budgets while our H₂O can still match the full cache performance with 20% budgets. In some tasks, our methods even surpass the baseline models, which demonstrates a regularization effect of our H₂O. For example, in {OPT-66B, RTE}, {OPT-30B, MathQA} and {GPT-NeoX-20B, XSUM}, our H₂O achieves an extra performance improvement of 0.73%, 0.64% and 0.18 with 20% KV cache budget, respectively. These consistent results validate the effectiveness of our H₂O framework.

Table 2: Results of different sparsification methods *w.* or *w.o.* H₂. Experiments are conducted with OPT-30B with 20% KV cache budget.

Models	COPA	OpenBookQA	PiQA	Winogrande
Full	85.00	43.20	78.51	70.24
Local <i>w.o.</i> H ₂	48.00	25.20	55.82	49.17
Local <i>w.</i> H ₂	84.00	43.00	78.45	69.06
Sparse Transformer (strided) <i>w.o.</i> H ₂	50.00	24.60	56.20	47.59
Sparse Transformer (strided) <i>w.</i> H ₂	83.00	42.60	78.24	69.61
Sparse Transformer (fixed) <i>w.o.</i> H ₂	61.00	23.80	58.60	49.88
Sparse Transformer (fixed) <i>w.</i> H ₂	76.00	41.40	77.80	64.96

Enhancing Baseline Techniques. Importantly, we observe other sparsification baselines fail under an extremely low cache budget while combining the most recent KV embeddings with the ones of heavy hitters successfully achieves comparable performance as using full KV embeddings. From Table 2, we can observe that both "strided" and "fixed" sparse attention fail under 20% KV cache budgets, encountering a significant performance drop (up to 35% compared with the full cache). After combining with H₂, both approaches reach a similar performance as using full KV embeddings.

5.2 Heavy Hitter for High-Throughput Generative Inference

Table 3: Generation throughput (token/s) on a T4 GPU with different systems. In the sequence length row, we use "512 + 32" to denote a prompt length of 512 and a generation length of 32. "OOM" means out-of-memory. The gray text in the bracket denotes the effective batch size and the lowest level of the memory hierarchy that the system needs for offloading, where "C" means CPU and "G" means GPU.

Seq. length	512+32		512+512		512+1024	
	6.7B	30B	6.7B	30B	6.7B	30B
Accelerate	20.4 (2, G)	0.6 (8, C)	15.5 (1, G)	0.6 (8, C)	5.6 (16, C)	0.6 (8, C)
DeepSpeed	10.2 (16, C)	0.6 (4, C)	9.6 (16, C)	0.6 (4, C)	10.1 (16, C)	0.6 (4, C)
FlexGen	20.2 (2, G)	8.1 (144, C)	16.8 (1, G)	8.5 (80, C)	16.9 (1, G)	7.1 (48, C)
H ₂ O (20%)	35.1 (4, G)	12.7 (728, C)	51.7 (4, G)	18.83 (416, C)	52.1 (4, G)	13.82 (264, C)

Table 5: Generation throughput and latency on an A100 GPU. In the sequence length row, we use “7000 + 1024” to denote a prompt length of 7000 and a generation length of 1024. “OOM” means out-of-memory.

Seq. length	Model size	Batch size	Metric	FlexGen	H ₂ O (20%)
7000+1024	30B	1	latency (s)	57.0	50.4
5000+5000	13B	4	latency (s)	214.2	155.4
2048+2048	6.7B	24	latency (s)	99.5	53.5
2048+2048	6.7B	24	throughput (token/s)	494.1	918.9
2048+2048	6.7B	64	throughput (token/s)	OOM	1161.0

We implement our KV cache eviction policy in a state-of-the-art inference engine, FlexGen [19], and report the throughput and latency improvements. H₂O is orthogonal to existing optimizations in FlexGen, such as offloading and quantization, so they can be combined to achieve better performance.

Setup We conducted experiments on two GPUs: an NVIDIA T4 (16GB) GPU and an NVIDIA A100 (80GB) GPU. On the T4 GPU, we evaluate the generation throughput following the settings in the FlexGen paper. The evaluated models are OPT-6.7B and OPT-30B. When the model and KV cache do not fit into a single GPU, we turn on CPU offloading.

The results of both pure GPU and GPU with CPU offloading are reported. All the speedup results are tested in an end-to-end setting, including both the pre-filling and generation phases. And it includes the time for constructing the H₂O KV cache. We use synthetic datasets where all prompts are padded to the same length. The system is then required to generate the same number of tokens for each prompt. We test different combinations of prompt and generation lengths. We also test our method on real-world datasets (XSUM) for further assessment. The evaluation metric is generation throughput, which is the number of generated tokens / (prompt time + decoding time). We use DeepSpeed ZeRO-Inference [17], Hugging Face Accelerate [18], and FlexGen [19] as baselines. On the A100 GPU, with more GPU memory, we evaluate the performance of the systems with sequence lengths up to 10K. Although OPT is only trained on 2K sequence length, we benchmark the throughput and latency performance to show the potential of H₂O for better models in the future.

Results. Table 3& 4 shows the generation throughput of all systems on the T4 GPU. With our KV cache eviction policy, the memory usage is reduced, which brings two advantages: 1) we can use a much larger batch size; 2) we can make a setting from requiring offloading to not requiring offloading. As shown in Table 3& 4, H₂O with a 20% budget improves the generation throughput over FlexGen, DeepSpeed, and Accelerate by up to 3 $\times$, 29 $\times$, and 29 $\times$, respectively, across both synthetic and real-world dataset. The results on the A100 GPU with sequence lengths from 4K to 10K are listed in Table 5. With the same batch size, H₂O can reduce the latency by 1.1 – 1.9 $\times$ compared to FlexGen. Additionally, H₂O saves memory so it allows a larger batch size, which brings 2.3 $\times$ improvement on generation throughput for OPT-6.7B.

5.3 Ablation Results

We present extensive ablation studies of H₂O on (1) infinite-length input, (2) different number of shots, (3) compatibility with quantization methods on KV cache, and (4) dissecting the effectiveness of different components. We find a surprising property of H₂O – it not only improves the efficiency of LLMs, but also increases the diversity of the generated text.

Q1: Can H₂O empower LLMs to process infinite-length inputs? A1: Effective generation with sequence length up to four million tokens. Some recent works [52, 53] demonstrate the possibility of handling infinite-length inputs, a notable challenge in current LLMs. These methods employ an attention sink that retains the first few tokens and applies position rolling in the KV cache, empowering LLMs to process infinite-length inputs. Inspired by this progress, we further implement our H₂O for infinite-length inputs. Figure 5 showcases the positive results of H₂O, *i.e.*, H₂O can empower LLMs to tackle input with length up to four mil-

Table 4: Results of generation throughput (token/s) on a T4 GPU with different systems on real-world datasets, XSUM.

Model size	6.7B	30B
Accelerate	11.98 (1, G)	0.23 (2, C)
DeepSpeed	3.52 (6, C)	0.31 (2, C)
FlexGen	10.80 (1, G)	3.29 (44, C)
H ₂ O (20%)	30.40 (1, G)	6.70 (180, C)